

Predicting UFC Fight Outcomes Using Pre-Fight Fighter Statistics

Compulsory Exercise 2 — TMA4268 Statistical Learning V2026

Ricardo Sonda Guiraudeli; Vegard Ragnhildstveit; Anders Bekkevold

April 2026

Abstract

We examine the feasibility of predicting the winner of a UFC martial arts fight from pre-fight statistics only. The target is a binary outcome: whether the fighter assigned to the Red corner wins ($\text{WinnerRed} = 1$) or loses ($\text{WinnerRed} = 0$). The dataset is the *Ultimate UFC Dataset* sourced from Kaggle (`mdabbert/ultimate-ufc-dataset`), containing 6,528 bouts between March 2010 and December 2024 with per-fighter cumulative statistics recorded at fight time.

We make use of a *difference features* feature set - each numeric statistic is replaced by its Red-minus-Blue signed difference - to remove redundancy in the dataset and reduce dimensionality from 118 to 26 predictors. Three model families are compared: penalised logistic regression (Elastic Net, tuned with nested cross-validation), a Random Forest ensemble, and gradient-boosted trees (XGBoost). All models are evaluated on a chronological 80/20 holdout split to prevent time leakage, and benchmarked against the implied win probabilities derived from pre-fight betting odds.

The bookmaker (betting-implied) baseline achieves an accuracy of 69.6% and a ROC AUC of 0.743. The model with best performance, the Random Forest, reaches an accuracy of 63.6% and a ROC AUC of 0.693. The gap to the bookmaker confirms that publicly available fighter statistics contain real predictive signal but fall well short of the full information encoded in market odds, as expected. Among observable predictors, the average takedown-landed difference, reach advantage, and win-streak difference emerge as the strongest predictors of fight outcome, while a negative age coefficient reveals that an age gap is a significant disadvantage for older fighters.

1 Introduction

Mixed martial arts (MMA) is one of the fastest-growing combat sports globally, and the UFC is its premier organisation. Fights can be thought of as a highly complex event influenced by physical attributes, technical skills, recent form, and matchup dynamics. Predicting fight outcomes is a classic classification problem: given observable pre-fight information about both fighters, can we assign a probability that the Red-corner fighter wins?

Dataset. We use the *Ultimate UFC Dataset* published by user `mdabbert` on Kaggle (`mdabbert/ultimate-ufc-dataset`). The dataset records 6,528 bouts with 118 raw columns, including per-fighter cumulative career statistics (wins, losses, streaks, striking and grappling averages, physical measurements), contextual fight information (weight class, number of rounds, gender), and bookmaker moneyline odds. After filtering to bouts with a decisive Red or Blue winner (no ties) and dropping the pre-computed difference columns provided by the source, we retain 6,275 bouts for analysis.

Task and scope. Our goal is to build a classifier that, given only pre-fight information, assigns a calibrated probability of a Red-corner victory. We benchmark every model against bookmaker-implied probabilities — a pretty-much-near-market-efficient baseline that absorbs information beyond public statistics. This project

is meant as an exercise in the statistical methods employed in the statistical learning course and we do not intend to create a model that beats the moneyline odds.

Practical constraint. Because leakage from betting odds onto odds-derived features would trivially dominate any model, odds-derived columns are excluded from all feature sets; the odds appear only as a benchmark in evaluation.

How moneyline odds work. UFC betting odds are usually quoted as *American moneyline* odds. A negative line (e.g., -150) indicates the favourite: you must stake 150 units to win 100 units of profit. A positive line (e.g., $+130$) indicates the underdog: a 100-unit stake wins 130 units of profit. These lines can be converted to implied win probabilities using $p = \frac{|o|}{|o|+100}$ for negative odds $o < 0$, and $p = \frac{100}{o+100}$ for positive odds $o > 0$. In practice, bookmakers include a margin (“vig”), so the two implied probabilities in a matchup typically sum to slightly more than 1. In this project, we generally work with implied probabilities instead of the moneyline odds.

2 Descriptive Data Analysis

2.1 Dataset overview

After cleaning, the dataset contains 6,275 bouts spanning March 2010 to December 2024. The outcome is moderately imbalanced: 58.1% of bouts result in a Red-corner victory, as shown in Figure 1. This imbalance reflects the UFC’s convention of assigning the Red corner to champions and higher-ranked fighters, who naturally win more often. Table 1 summarises the key numeric predictors used in modelling.

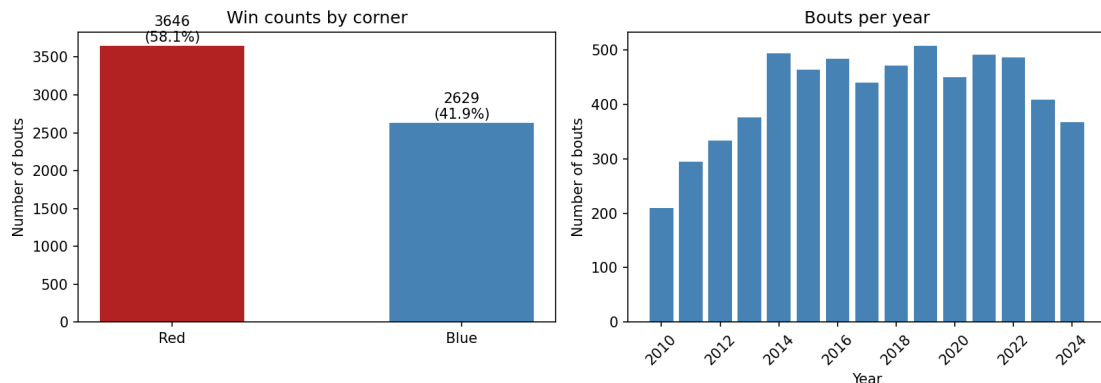


Figure 1: Left: Red vs Blue win counts in the cleaned dataset (6,275 bouts). Red-corner fighters win 58.1% of bouts. Right: number of bouts per year; data coverage is near-complete from 2013 onwards.

Table 1: Summary statistics for selected difference features (*Red minus Blue*) on the full cleaned dataset ($n = 6,275$). Missing-value counts differ per feature because career statistics were not tracked for early bouts.

Feature	Mean	Std	Min	Max
AgeDiff	0.56	5.18	-16.0	17.0
ReachCmsDiff	0.23	8.62	-30.5	188.0
CurrentWinStreakDiff	0.14	1.87	-10.0	18.0
CurrentLoseStreakDiff	0.12	1.01	-5.0	6.0
AvgSigStrLandedDiff	0.32	17.4	-128.2	104.7
AvgSigStrPctDiff	0.01	0.14	-0.57	0.82
AvgTDLandedDiff	0.09	1.79	-10.9	11.0
WinsDiff	1.49	4.17	-23.0	28.0

The near-zero means for the difference features are expected, since the UFC broadly matches fighters of comparable ability, so the average edge is close to zero. The positive mean for `WinsDiff` (1.49) and `AgeDiff` (0.56) reflects the aforementioned policy for assigning more experienced fighters to the red corner.

2.2 Missingness

Many statistics columns have substantial missing values for early bouts during when tracking was incomplete. Also, a substantial number of features related to weight-classes are empty both due to the sparse encoding and due unrecorded data for unranked fighters. Figure 2 shows the completeness rate per column. The *diff* feature set strategy (Section 3) retains all rows and delegates imputation to the model pipeline.

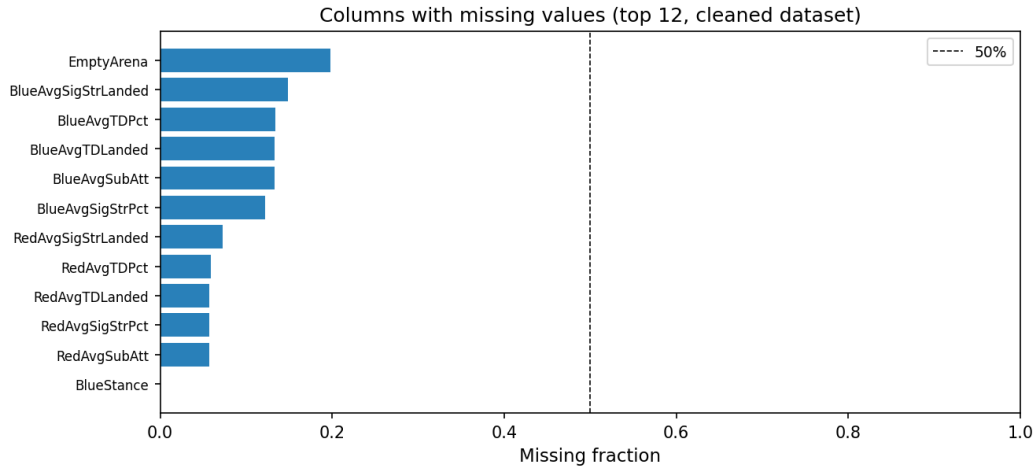


Figure 2: Fraction of non-missing values per column in the raw dataset. Fighter striking and grappling averages have notably lower completeness for older bouts.

2.3 Correlation with outcome

Figure 3 shows the Pearson correlation of each numeric raw column with the binary outcome `WinnerRed`. Correlations are modest ($|r| < 0.12$), confirming that no single feature is strongly predictive on its own. Age, win streak, cumulative wins, and significant-strike statistics show the highest marginal correlations.

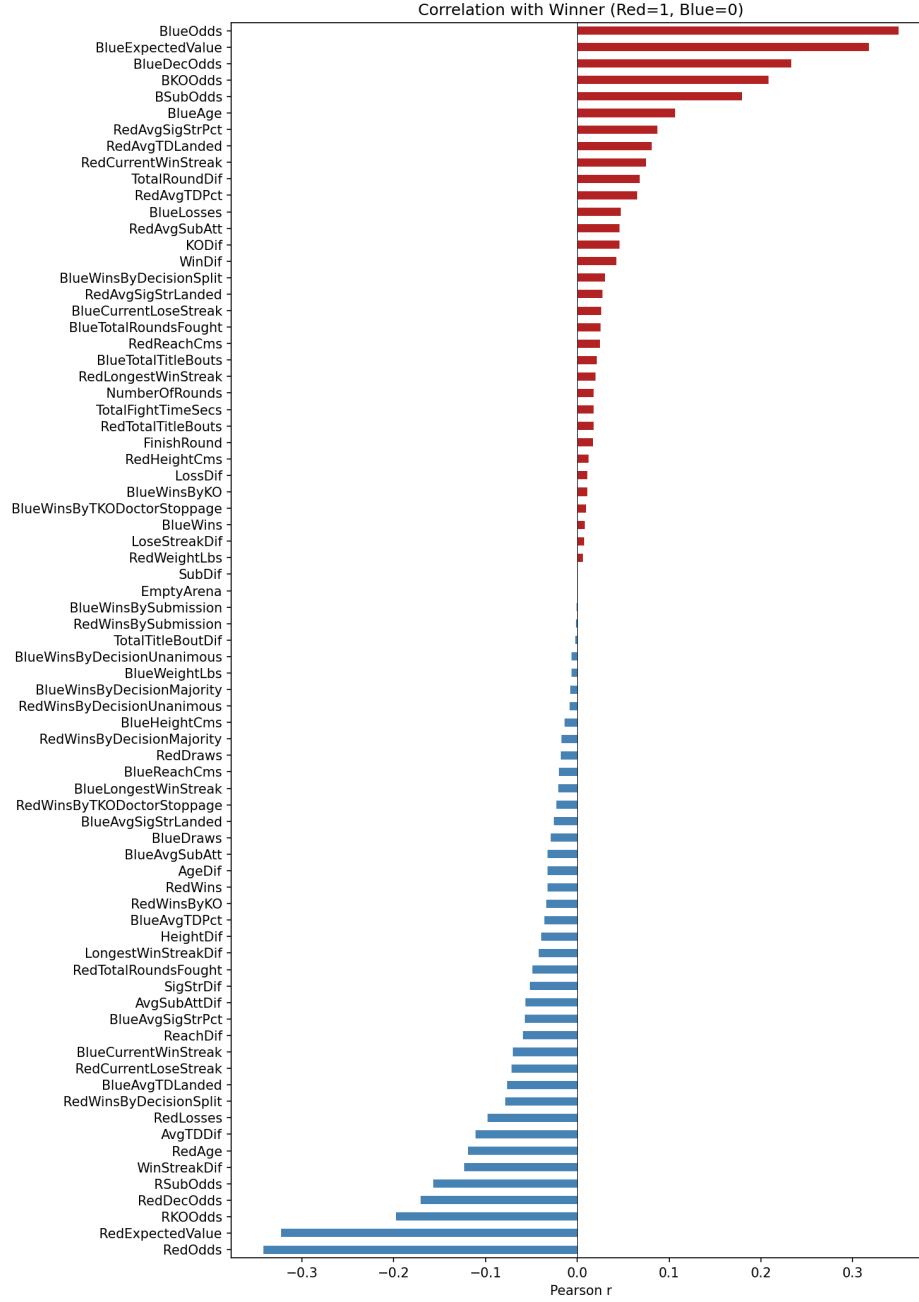


Figure 3: Pearson correlation of raw numeric columns with `WinnerRed`. The strongest individual predictors are win-streak and total-wins differences (Red corner), all with $|r| < 0.12$.

2.4 Feature distributions by outcome

Figure 4 shows violin plots of the six most important difference features split by fight outcome. The distributions largely overlap, which is consistent with the low individual correlations. The signal is subtle: in Red-win fights the win-streak and reach distributions shift slightly upward, confirming a real but weak effect.

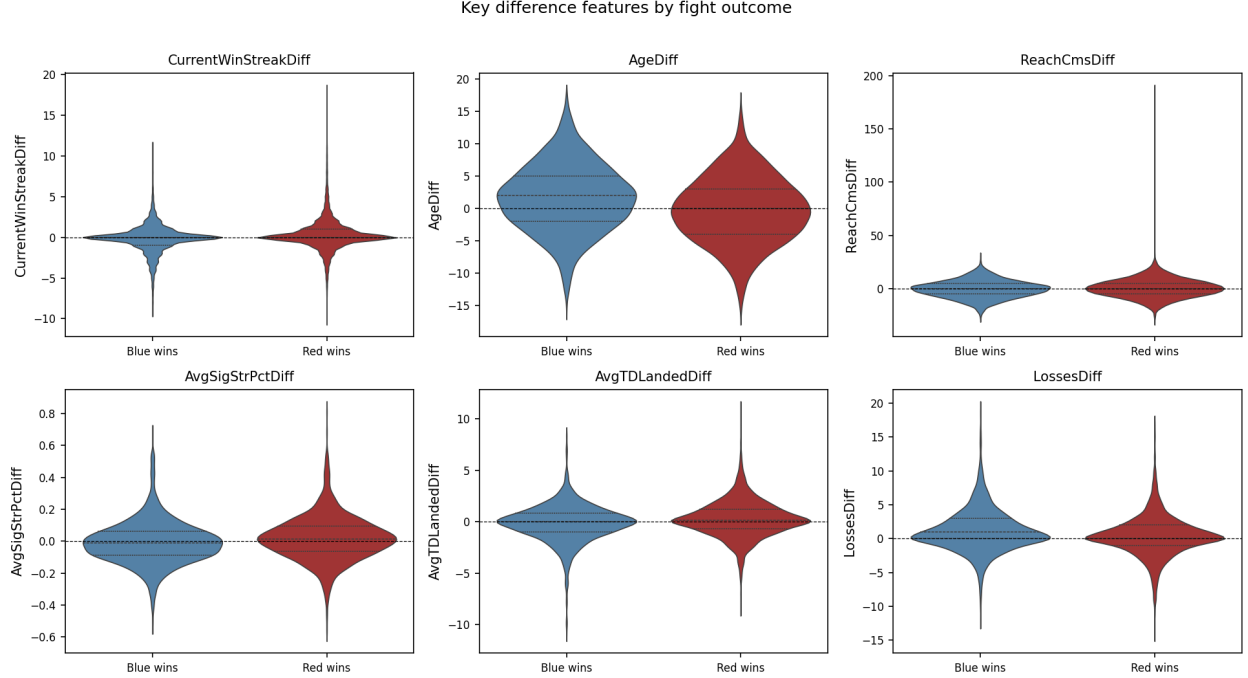


Figure 4: Distribution of key difference features (Red minus Blue) split by outcome. Dashed lines mark zero (even matchup). Distributions overlap substantially, reflecting the difficulty of the prediction task.

2.5 Feature correlation structure

The fighter statistics are highly internally correlated (e.g., wins is correlated with win streak and total rounds fought). This motivates using difference features to compress the Red/Blue column pairs into a single signed quantity, reducing some of the multicollinearity in the features.

3 Methods

3.1 Feature engineering

The raw dataset contains symmetrically paired Red/Blue columns for every fighter statistic. We replace each pair $(x_{\text{red}}, x_{\text{blue}})$ with the signed difference $\Delta x = x_{\text{red}} - x_{\text{blue}}$. This yields symmetric features: swapping Red and Blue negates all difference features and flips the outcome, so the representation is not biased on the corners. The resulting *diff* feature set contains 26 predictors (numeric differences plus categorical context variables: gender, title-bout indicator, and number of scheduled rounds).

Missing values are imputed inside using medians for numeric features and mode for categorical features, so all 6,275 bouts are preserved.

3.2 Data split

All models use a **chronological holdout split**: fights are sorted by date, the earliest 80% form the training set (5,020 bouts), and the most recent 20% form the test set (1,255 bouts; fights from approximately late 2021 onwards). This choice prevents *time leakage*: since some numeric features are cumulative fighter statistics, information about the results of past fights are contained in the later entries. Using this information to predict previous fights thus leads to leakage.

3.3 Model 1: Logistic Regression (baseline)

Logistic regression models the log-odds of Red winning as a linear function of the (min-max scaled and imputed) difference features:

$$\log \frac{P(\text{Red wins})}{1 - P(\text{Red wins})} = \beta_0 + \boldsymbol{\beta}^\top \Delta \mathbf{x}.$$

Strengths: interpretable coefficients, fast to fit, simple. **Weaknesses:** assumes a linear decision boundary; cannot capture non-linear interactions between features. In our setting the feature space is noisy and low-signal, so a linear boundary is a reasonable starting assumption. No regularisation is applied for the baseline model. Coefficients are estimated by likelihood.

3.4 Model 2: Elastic Net Logistic Regression (penalised, tuned)

An Elastic Net is just a Logistic Regression with hybrid L_1 and L_2 regularisation penalty to the log-likelihood:

$$\mathcal{L}(\boldsymbol{\beta}) - \lambda \left[\frac{1 - \alpha}{2} \|\boldsymbol{\beta}\|_2^2 + \alpha \|\boldsymbol{\beta}\|_1 \right],$$

where $C = 1/\lambda$ controls regularisation strength and α (called `l1_ratio` in scikit-learn) interpolates between Ridge ($\alpha = 0$) and Lasso ($\alpha = 1$). Elastic Net is preferable to pure Lasso when predictors are correlated: Lasso tends to arbitrarily discard one of a correlated group, whereas Ridge-like shrinkage stabilises the selection.

Our **hyperparameter tuning** uses a nested cross-validation structure:

- **Outer split:** the chronological 80/20 holdout (shared with all models).
- **Inner CV:** 5-fold stratified cross-validation over the training set.
- **Grid:** $C \in \{0.01, 0.1, 1.0, 10.0\}$, $\alpha \in \{0.1, 0.5, 0.9, 1.0\}$ (16 combinations).

The best parameters found were $C = 0.01$ (strong regularisation), $\alpha = 0.1$ (mostly Ridge with a light Lasso component). 21 out of 26 features survived the Lasso non-zero coefficients.

3.5 Model 3: Random Forest

A Random Forest trains $B = 400$ decision trees on bootstrap samples and averages their estimates. Each tree considers $\sqrt{p}$ randomly selected features at each split. **Strengths:** captures non-linear structure and feature interactions without explicit specification; robust to outliers; provides feature importances via mean decrease in impurity. **Weaknesses:** less interpretable than linear models; can overfit with deep trees on small datasets. We set `min_samples_leaf = 10` to regularise tree depth and reduce variance; this was chosen by our judgment given the dataset size (5,020 training observations, 26 features).

3.6 Model 4: Gradient Boosted Trees (XGBoost)

XGBoost builds trees sequentially, with each tree correcting the residuals of the current ensemble. The learning rate (`learning_rate = 0.05`) limits each tree's contribution to prevent overfitting, and stochastic subsampling (`subsample = 0.8`, `colsample_bytree = 0.8`) introduces variance reduction. The minimum child weight (`min_child_weight = 5`) acts as additional regularisation on leaf size. **Strengths:** strong empirical performance; handles missing values natively; flexible regularisation. **Weaknesses:** more hyperparameters to configure; prone to overfitting without careful tuning; less interpretable than logistic regression.

3.7 Evaluation metrics

All models are evaluated on the held-out chronological test set and compared against the bookmaker-implied probabilities. We report:

- **Accuracy:** fraction of correctly predicted outcomes (threshold at 0.5).

- **Log loss** (cross-entropy): measures calibration and sharpness of probability estimates. Lower is better. Treated as the primary ranking metric because it rewards well-calibrated probabilities, not just the sign of the prediction.
- **Brier score**: mean squared error between predicted probabilities and binary outcomes. Lower is better.
- **ROC AUC**: area under the receiver operating characteristic curve. Measures the discrimination ability independent of the decision threshold.

4 Results and Interpretation

4.1 Model performance

Table 2 compares all models on the 1,255-bout test set. The bookmaker benchmark is shown for reference. Δ values are signed differences relative to the bookmaker (for negative Δ log loss, Brier is better. For positive Δ accuracy, AUC is better).

Table 2: Test-set evaluation results (chronological holdout, $n_{\text{test}} = 1,255$).

Model	Acc	Δ Acc	Log Loss	Δ LL	Brier	ROC AUC
Bookmaker (baseline)	0.696	—	0.593	—	0.204	0.743
Naive (majority class)	0.571	−0.124	0.693	+0.100	0.250	0.500
Logistic Regression	0.635	−0.060	0.636	+0.043	0.223	0.673
Elastic Net (nested CV)	0.640	−0.056	0.635	+0.041	0.222	0.682
Random Forest	0.636	−0.060	0.633	+0.040	0.221	0.693
XGBoost	0.632	−0.064	0.636	+0.043	0.223	0.670

Random Forest achieves the best log loss (0.633), Brier score (0.221), and ROC AUC (0.693). However, the differences among Logistic Regression, Elastic Net, and Random Forest are small (log loss range < 0.004), suggesting that the dominant bottleneck is the limited content of the public statistics, not the model family. XGBoost performs slightly worse despite its higher complexity.

All models comfortably beat the naive majority-class baseline (AUC 0.5) but remain well below the bookmaker (AUC 0.743). One way to interpret the persistent ≈ 5.5 percentage-point accuracy gap to the bookmaker is that it quantifies the information advantage of market prices over publicly available fight statistics.

4.2 Confusion matrices

Table 3 shows confusion matrix entries and precision/recall for each model. A common pattern is that all models predict Red wins the large majority of the time, driven by the 58.1% class imbalance. Recall for the Red class (fraction of actual Red wins correctly called) is above 0.83 for all linear/ensemble models, while recall for the Blue class is much lower. XGBoost achieves the most balanced predictions (TN = 233, highest among all models), reflecting the effect of `min_child_weight` regularisation.

Table 3: Confusion matrix entries and precision/recall on the test set ($n = 1,255$; 717 Red wins, 538 Blue wins). Precision and recall are reported for the Red-wins class (positive class).

Model	TN	FP	FN	TP	Precision	Recall
Logistic Regression	202	336	122	595	0.639	0.830
Elastic Net (nested CV)	198	340	112	605	0.640	0.844
Random Forest	201	337	120	597	0.639	0.833
XGBoost	233	305	157	560	0.647	0.781

4.3 ROC curves

Figure 5 shows the ROC curve for the best-performing model (Random Forest) alongside the bookmaker benchmark. The bookmaker ROC curve lies clearly above the model curve across all thresholds, confirming that market odds encode more discriminative information than our feature set.

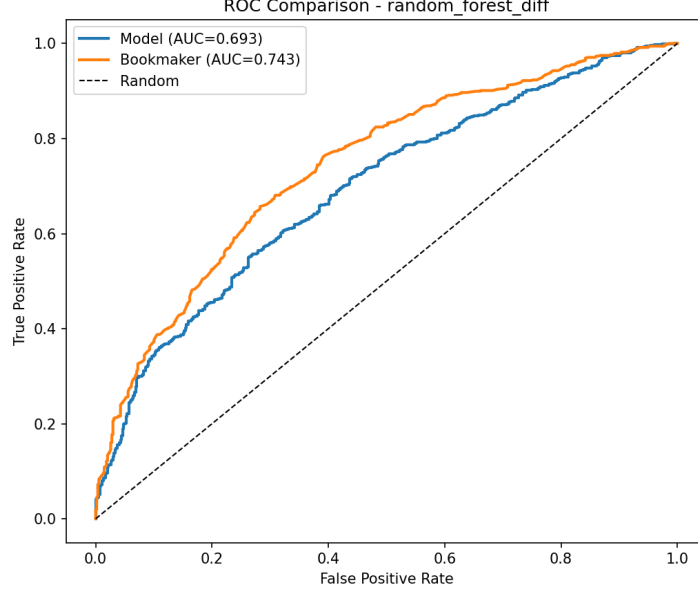


Figure 5: ROC curves for the Random Forest model versus the bookmaker baseline on the test set. The bookmaker (AUC = 0.743) outperforms the best statistical model (AUC = 0.693).

4.4 Coefficient interpretation (Elastic Net)

Table 4 shows the largest-magnitude Elastic Net coefficients after nested-CV tuning ($C = 0.01$, $\alpha = 0.1$). The strong regularisation ($C = 0.01$) shrinks coefficients toward zero, and 21 out of 26 features survive with non-zero weights. Positive coefficients indicate features that, when larger for the Red corner, increase the predicted probability of a Red victory.

Table 4: Top 8 Elastic Net coefficients (sorted by $|\hat{\beta}|$)

Feature	$\hat{\beta}$	Interpretation
AgeDiff	-0.259	Older Red fighter $\rightarrow$ lower Red win probability
AvgTDLandedDiff	+0.123	Red takedown advantage $\rightarrow$ higher Red win prob.
ReachCmsDiff	+0.105	Red reach advantage $\rightarrow$ higher Red win prob.
CurrentWinStreakDiff	+0.104	Longer Red active win streak $\rightarrow$ higher Red win prob.
LossesDiff	-0.087	More career losses for Red $\rightarrow$ lower Red win prob.
WeightLbsDiff	+0.080	Red weight advantage $\rightarrow$ higher Red win prob.
LongestWinStreakDiff	+0.069	Longer Red peak win streak $\rightarrow$ higher Red win prob.
DrawsDiff	+0.057	More draws for Red $\rightarrow$ slight Red win advantage

The negative age coefficient warrants attention: $\text{AgeDiff} = \text{Age}_{\text{Red}} - \text{Age}_{\text{Blue}}$, so the negative sign means older Red fighters are predicted to win less often. Since Red fighters tend to be older (mean $\text{AgeDiff} = 0.56$ years), this coefficient captures a pattern where relatively younger opponents outperform statistical expectations, possibly reflecting some age related decline of the more veteran Red fighters.

4.5 Feature importance (Random Forest)

Figure 6 shows the top-20 feature importances from the Random Forest, measured by mean decrease in impurity. The top three features are `AgeDiff`, `AvgSigStrLandedDiff`, and `AvgTDLandedDiff`. This broadly agrees with the Elastic Net coefficient magnitudes (Table 4), giving confidence that these features carry genuine signal rather than being artefacts of one particular model class.

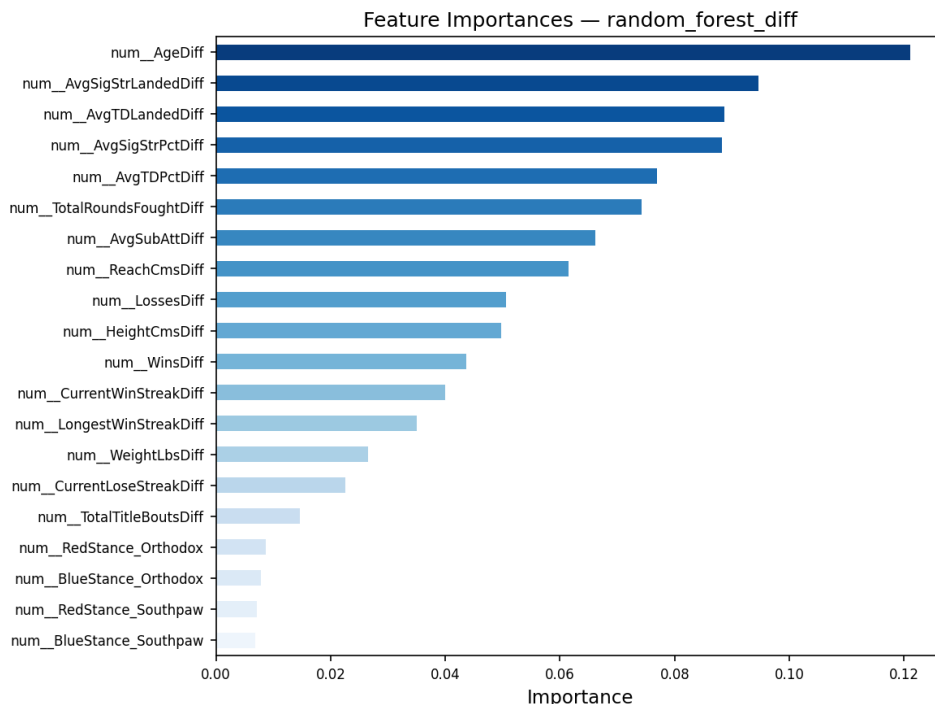


Figure 6: Random Forest feature importances (mean decrease in impurity) for the top 20 features. `AgeDiff` dominates, with grappling and striking averages following.

4.6 Bias-variance discussion

Logistic regression has the lowest variance but a linear decision boundary that cannot capture threshold effects (e.g., a reach advantage may matter very differently in the heavyweight division versus flyweight). Random Forest and XGBoost have lower bias and can fit interactions, but their higher variance must be controlled through regularisation (`min_samples_leaf` = 10 and `min_child_weight` = 5). Despite the added flexibility, ensemble methods do not substantially improve over regularised logistic regression (log loss difference < 0.004). This suggests that the signal-to-noise ratio in public UFC statistics is intrinsically low, and the variance cost of flexible models nearly cancels their bias benefit.

The fact that $C = 0.01$ (strong regularisation) was selected by inner CV for the Elastic Net also points in this direction: the CV is protecting against overfitting to noise, not trying to fit a complex signal.

4.7 Limitations

- **Bookmaker gap:** The persistent ≈ 5.5 percentage-point accuracy gap to the bookmaker suggests important predictors (training camp quality, recent injuries, travel, stylistic matchups) are unobservable from public statistics.
- **Missing data:** Many earlier bouts lack complete fighter statistics and median imputation introduces noise that may weaken model discrimination for these fights.

- **Label asymmetry:** The Red/Blue assignment is non-random as Red fighters tend to be higher-ranked and older. This means the difference features carry a structural bias toward the Red corner that the model must implicitly correct for.
- **Fixed hyperparameters:** Hyperparameters for Random Forest and XGBoost were not formally tuned.
- **Chronological split:** Using only the last 20% of fights as the test set means test performance is measured on recent fights. If fighter statistics tracking or matchmaking strategy has changed over time, the train-test gap may reflect a distributional shift rather than a true generalisation error.

5 Summary

We framed UFC fight outcome prediction as a binary classification problem and compared three model families on a chronological test set of 1,255 bouts. All models comfortably beat the naive majority-class baseline (AUC 0.5) and extract meaningful signal from the difference feature representation. No model closes the gap to the bookmaker benchmark (AUC 0.743), confirming that market-implied probabilities encode information not accessible from public fighter statistics alone.

The Random Forest achieved the best overall performance (log loss 0.633, AUC 0.693), but the margin over regularised logistic regression is small. The Elastic Net with nested cross-validation provided the best interpretability: inner cross-validation selected strong regularisation ($C = 0.01$), and 21 out of 26 features survived the Lasso step. The most important predictors across both model families were average takedowns landed, age difference, reach advantage, and win-streak difference.

The most practically significant finding is the consistency of the bookmaker gap across all models. It establishes a clear ceiling: the information contained in publicly available cumulative career statistics is real but insufficient, and closing the gap would likely require access to information that is not neatly available in some public dataset.

Declaration of AI Usage

This project made use of AI coding assistants, specifically Claude, during the development process. The scope and boundaries of that usage are described below.

Work performed by the human authors

All intellectual and analytical work in this project was carried out by the human authors. This includes:

- **Problem formulation.** Choosing the UFC dataset, framing the task as binary classification of the Red-corner winner, and deciding to benchmark against bookmaker-implied probabilities.
- **EDA.** Investigating the class imbalance (58.1% Red wins), identifying the Red-corner assignment convention, examining missingness patterns, analysing the correlation structure among fighter statistics, and interpreting the violin-plot distributions.
- **Feature engineering decisions.** Designing the *difference features* representation ($\Delta x = x_{\text{Red}} - x_{\text{Blue}}$), reasoning about its symmetry properties, deciding which contextual columns to retain (gender, title bout, number of rounds), and choosing to exclude odds-derived columns from the feature set to prevent leakage.
- **Model selection.** Choosing the three model families (logistic regression, Random Forest, XG-Boost) and the Elastic Net variant with nested cross-validation. Selecting hyperparameter grids, deciding on the regularisation strategy, and setting tree-depth constraints (`min_samples_leaf`, `min_child_weight`).
- **Evaluation design.** Designing the chronological 80/20 train-test split to prevent time leakage, selecting evaluation metrics (log loss, Brier score, ROC AUC, accuracy), and framing the bookmaker as the primary benchmark.
- **Interpretation of results.** Interpreting Elastic Net coefficients (e.g. the negative age coefficient and its implications), reading Random Forest feature importances, analysing confusion matrices, discussing the bias-variance trade-off across model families, and drawing conclusions about the bookmaker gap.
- **Report writing.** Drafting, structuring, and revising the full latex report, including the abstract, methodology descriptions, results discussion, and limitations section. However, AI was also used liberally for drafting and formatting the .tex file for the report. This includes rewording of paragraphs and the like.

In short, all decisions that require understanding of statistical learning - choosing features, selecting and configuring models, designing the evaluation protocol, and interpreting the output - were done by the authors.

Work assisted by AI

The AI assistant was used primarily as a *coding tool* for implementation tasks that do not involve statistical reasoning. Typical prompts included instructions such as:

- Plotting and formatting: “change the colour of this graph”, “create violin plots on [X, Y, Z] features”, “add axis labels and a legend”.
- Pipeline assembly: “make a pipeline with min-max scaling and median imputation”, “wrap this in a `ColumnTransformer`”.
- Code scaffolding: “generate a confusion matrix plot from these predictions”, “save the metrics as JSON”.
- Repository organisation: setting up the `Makefile`, configuring directory paths, managing imports, and structuring the Python package layout.

These are tasks where the human author specified *what* to produce and the AI translated that specification into syntactically correct code. The AI did not choose which plots to make, which models to train, which features to include, or how to interpret any result.